

Investigación sobre Metodologías Ágiles: Contexto, Análisis y Comparativa entre Scrum, XP, Kanban y Crystal

Juan Diego Contreras Yepes 20211020069

Ingeniería de Sistemas

Universidad Distrital Francisco José de Caldas

Bogotá, Colombia

Resumen—El paradigma del desarrollo de software ha evolucionado significativamente con la adopción de metodologías ágiles, las cuales priorizan la adaptabilidad, la entrega continua de valor y la colaboración humana por encima de la planificación rígida y predictiva. Este artículo presenta un contexto histórico y conceptual de las metodologías ágiles, seguido de un análisis investigativo de cuatro marcos de trabajo representativos: Scrum, eXtreme Programming (XP), Kanban y la familia Crystal. Se describen sus características fundamentales y se incluye una matriz comparativa que facilita la selección de la metodología adecuada según el contexto del proyecto.

Index Terms—Metodologías Ágiles, Scrum, eXtreme Programming (XP), Kanban, Crystal, Ingeniería de Software.

I. INTRODUCCIÓN Y CONTEXTO HISTÓRICO

Durante las últimas décadas, la ingeniería de software dependía fuertemente de metodologías tradicionales o enfoques predictivos, como el modelo en cascada (Waterfall). Estos modelos exigían la definición exhaustiva de requisitos desde las fases iniciales, seguidas de etapas secuenciales de diseño, implementación y pruebas. Sin embargo, la alta volatilidad de los requisitos del cliente y la rápida evolución tecnológica evidenciaban la rigidez de estos modelos, volviéndolos propensos a sobrecostos, retrasos y a la entrega de productos que no satisfacían las necesidades reales del usuario final.

En respuesta a esta crisis, en febrero del año 2001, un grupo de diecisiete expertos en desarrollo de software se reunió en Utah, Estados Unidos, para redactar el denominado *Manifiesto Ágil*. Este documento sentó las bases de una nueva filosofía basada en cuatro valores fundamentales:

1. **Individuos y sus interacciones** por encima de procesos y herramientas.
2. **Software funcionando** por encima de documentación exhaustiva.
3. **Colaboración con el cliente** por encima de la negociación contractual.
4. **Respuesta ante el cambio** por encima de seguir un plan estricto.

A partir de estos valores, se consolidaron diversas metodologías que, aunque difieren en sus tácticas y artefactos, comparten el objetivo de entregar valor de forma iterativa e incremental.

II. MARCOS DE TRABAJO ÁGILES

II-A. Scrum

Scrum es, en la actualidad, el marco de trabajo ágil más adoptado a nivel global. Se define como un proceso empírico y heurístico que permite a los equipos abordar problemas complejos adaptativos, entregando productos de máximo valor de forma iterativa.

Características Principales:

- **Roles Definidos:** Scrum establece tres responsabilidades clave. El **Product Owner** maximiza el valor del producto y gestiona el *Product Backlog*; el **Scrum Master** es un líder servicial que asegura la correcta aplicación del marco; y los **Developers** son los profesionales que construyen el incremento del producto.
- **Iteraciones (Sprints):** El desarrollo se divide en ciclos de duración fija (timeboxes) llamados Sprints, que suelen durar entre una y cuatro semanas.
- **Ceremonias:** Incluye eventos estructurados para la inspección y adaptación: Sprint Planning, Daily Scrum, Sprint Review y Sprint Retrospective.

II-B. eXtreme Programming (XP)

Creada por Kent Beck a finales de la década de 1990, eXtreme Programming (XP) es una metodología centrada exclusivamente en la ingeniería de software. Su objetivo es llevar las mejores prácticas de programación a niveles “extremos” para garantizar un código de alta calidad y un diseño robusto frente a los cambios.

Características Principales:

- **Desarrollo Guiado por Pruebas (TDD):** Exige que las pruebas unitarias y de integración se escriban antes que el código de producción.
- **Programación en Parejas (Pair Programming):** Dos desarrolladores comparten una misma estación de trabajo; uno escribe el código (piloto) mientras el otro revisa estratégicamente el diseño (navegante).
- **Integración Continua:** Los cambios de código se integran en un repositorio compartido múltiples veces al día, ejecutando pruebas automáticas para detectar conflictos de forma temprana.

Tabla I
MATRIZ COMPARATIVA DE METODOLOGÍAS ÁGILES

Metodología	Enfoque Principal	Cadencia / Ciclo	Tamaño Ideal de Equipo	Prácticas Distintivas
Scrum	Gestión de proyectos, adaptación empírica y entrega de valor constante.	Iterativo (Sprints de 1 a 4 semanas).	3 a 9 personas por equipo (escalable).	Roles estrictos, Product Backlog, Daily Stand-up, Retrospectivas.
XP (eXtreme Programming)	Excelencia técnica y calidad del código fuente.	Iterativo (Frecuencias muy cortas, 1 a 2 semanas).	Equipos pequeños (2 a 10 personas).	Pair programming, TDD, integración continua, refactorización.
Kanban	Optimización del flujo de trabajo y eliminación de cuellos de botella.	Flujo continuo (sin iteraciones fijas de tiempo).	Flexible, adaptable a cualquier tamaño.	Tablero visual, límites de WIP (Work In Progress), métricas de Lead/Cycle Time.
Crystal	Comunicación humana, tolerancia, seguridad y adaptabilidad por criticidad.	Adaptable según el color de la metodología elegida.	Desde 2 hasta equipos corporativos grandes.	Clasificación por colores según tamaño y riesgo (Clear, Yellow, Orange, etc.).

- **Diseño Simple y Refactorización:** Se implementa la solución más sencilla que funcione, mejorando continuamente la estructura interna del código sin alterar su comportamiento externo.

II-C. Kanban

A diferencia de Scrum y XP, Kanban (término japonés que significa “tarjeta visual”) no es un marco de trabajo iterativo, sino un método de flujo continuo derivado del sistema de producción de Toyota (Lean Manufacturing). Su propósito es gestionar el trabajo equilibrando la demanda con la capacidad disponible del equipo.

Características Principales:

- **Gestión Visual del Flujo:** Emplea un tablero físico o digital dividido en columnas (e.g., Backlog, Análisis, Desarrollo, Pruebas, Despliegue) donde las tarjetas representan las tareas que fluyen por el sistema.
- **Límites WIP (Work in Progress):** Constituyen la regla más crítica de Kanban. Limitan la cantidad máxima de tareas que pueden estar activas en una columna simultáneamente. Esto previene la sobrecarga cognitiva y expone los cuellos de botella.
- **Sistema Pull:** Las tareas no se “empujan” hacia la siguiente fase, sino que se “jalan” cuando hay capacidad disponible, asegurando una entrega continua y fluida.

II-D. Familia Crystal

Desarrollada por Alistair Cockburn, Crystal no es una metodología única, sino una familia de metodologías. Cockburn sostiene que los proyectos de software son “juegos cooperativos de invención y comunicación”, por lo que no existe un proceso universalmente aplicable.

Características Principales:

- **Enfoque Humano:** Crystal da mayor prioridad a la comunicación osmótica (escuchar en segundo plano la información del equipo) y a la interacción humana que a las herramientas o procesos estrictos.
- **Clasificación por Colores:** Las metodologías se categorizan en colores según el tamaño del equipo y la criticidad del sistema (el daño potencial si el software falla). Por ejemplo:

- **Crystal Clear:** Equipos de 1 a 6 personas; reglas muy ligeras.
- **Crystal Yellow:** Equipos de 7 a 20 personas.
- **Crystal Orange/Red:** Para equipos grandes (20-80+ personas), requiriendo mayor rigor en la documentación y la coordinación.

III. CONCLUSIONES

La adopción de una metodología ágil no debe ser una decisión arbitraria. Como se evidenció en la Tabla I, cada marco ofrece soluciones a problemáticas distintas. Scrum aporta un excelente marco organizativo; XP garantiza la sostenibilidad técnica del código a largo plazo; Kanban proporciona un control estadístico del flujo de trabajo ideal para equipos de soporte; y Crystal ofrece una escalabilidad estructurada basada en el factor humano.

En la ingeniería de sistemas contemporánea, es común que los equipos apliquen enfoques híbridos, como *Scrumban* (combinación de Scrum y Kanban) o la adopción de las prácticas técnicas de XP dentro de un esquema organizativo de Scrum, maximizando así la resiliencia y eficiencia en el ciclo de vida del software.

REFERENCIAS

- [1] K. Beck *et al.*, “Manifiesto for Agile Software Development,” 2001. [Online]. Disponible en: <http://agilemanifesto.org/>(<http://agilemanifesto.org/>)
- [2] K. Schwaber y J. Sutherland, “La Guía de Scrum,” Scrum.org, 2020.
- [3] K. Beck y C. Andres, *Extreme Programming Explained: Embrace Change*, 2.^a ed. Addison-Wesley Professional, 2004.
- [4] D. J. Anderson, *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010.
- [5] A. Cockburn, *Crystal Clear: A Human-Powered Methodology for Small Teams*. Addison-Wesley Professional, 2004.